WAF PROJECT · ARCHITECTURE MIGRATION

Worklet 2 Refactoring Guide

v1.0 Complete migration manual for converting Worklet 2 from the chained pipeline architecture to the gateway-orchestrated pattern. Covers file-by-file changes, before/after code examples, and the stdin/stdout subprocess integration contract.

PREPARED FOR	ISSUED BY	REVIEW DATE
Worklet 2 Team (C++)	W3 / Gateway Team	October 2026

Table of Contents

1. Executive Summary **2**

2. Architecture Comparison: Old vs New **3**

3. Required Code Changes - File by File **5**

4. Change 1 - Remove W3 Callback from Worklet2Gateway **6**

5. Change 2 - Remove Action Extraction & sessionId Handling **7**

6. Change 3 - Simplify ValidationResult to Boolean Output **8**

7. Change 4 - Create Standalone Executable (w2_main.cpp) **9**

8. Integration Contract - stdin / stdout Protocol **12**

9. Java Gateway Caller - Reference Implementation **14**

10. Migration Steps and Timeline **17**

11. What to Keep and What to Delete **18**

12. Testing and Acceptance Criteria **19**

1. Executive Summary

Worklet 2 (W2) currently implements a **chained pipeline** architecture in which it validates JSON structure, extracts the semantic action, and then directly calls Worklet 3 (W3) via a callback delegate. While W2's CYK engine and grammar handling are mathematically sound, the architectural role W2 has assumed creates tight coupling, complicates testing, and forces W3 to expose an HTTP endpoint even though both W1 and W3 are Java components that could communicate in-memory.

This guide defines the migration of W2 from the chained pipeline pattern to a **gateway-orchestrated** pattern. Under the new architecture, a central Java gateway (WAFProxyServer) intercepts every incoming request, calls W1 in-memory, spawns the W2 C++ executable as a subprocess to perform pure structural validation, calls W3 in-memory for workflow validation, and only then forwards the original request to the backend. W2 becomes a stateless, single-purpose structural validator that returns a boolean via stdout.

The migration requires six concrete changes to the W2 codebase. None of these changes affect W2's core CYK algorithm, grammar definition, or security pipeline (token limits, depth checks). Only the **boundary** of W2 changes - what it accepts as input, what it returns as output, and which downstream components it is allowed to talk to.

The Six Required Changes

#	Change	Effort	Risk
1	Remove Worklet3Delegate callback from Worklet2Gateway	Low	Low
2	Remove action extraction and HTTP method handling	Low	Low
3	Remove sessionId parameter from validateAndProcess()	Low	Low
4	Simplify ValidationResult struct to boolean output	Medium	Medium
5	Create w2_main.cpp standalone executable	Medium	Low
6	Test with Java subprocess caller	Medium	Low

Estimated total effort: 3 days. The CYK engine, grammar definitions, CNF converter, and structural validator core pipeline remain untouched. Only the gateway boundary and main entry point are refactored.

2. Architecture Comparison: Old vs New

The single most important architectural decision in this migration is the move from a chained pipeline (where each worklet calls the next) to an orchestrator pattern (where a central gateway calls each worklet). The diagram below shows the data flow under both architectures.

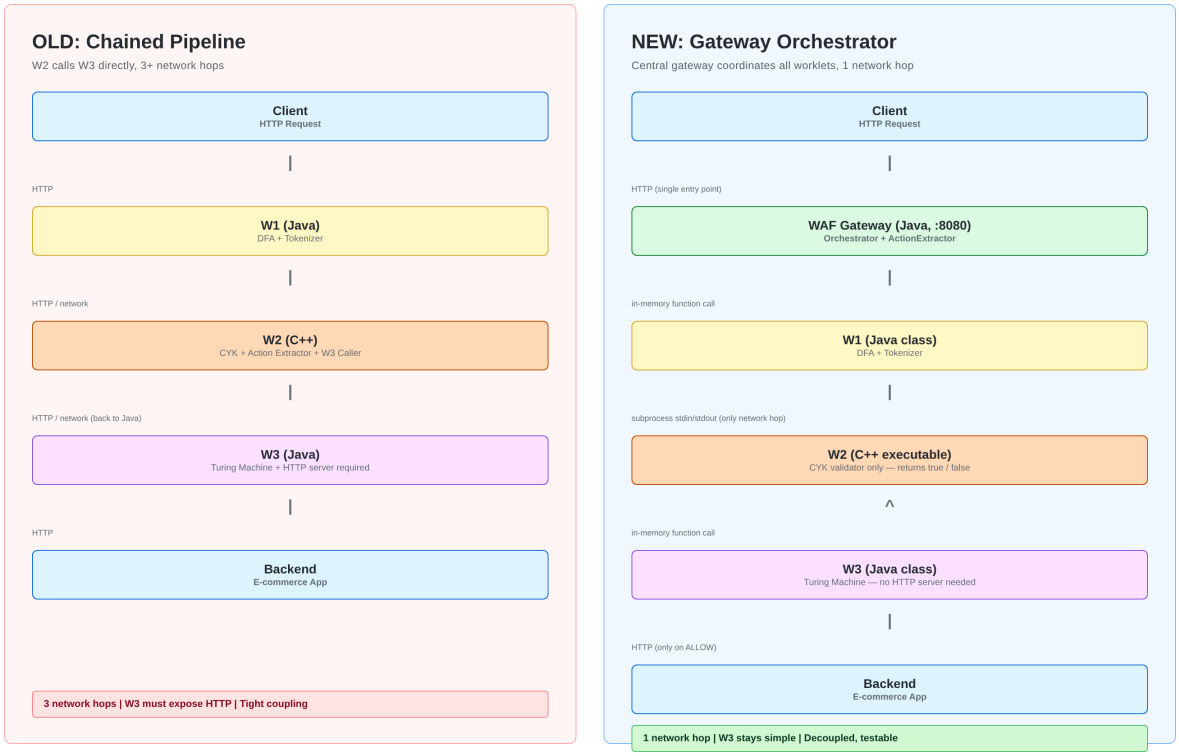


Figure 1. Old chained pipeline (left) vs new gateway-orchestrated architecture (right).

Quantitative Comparison

Metric	Old: Chained Pipeline	New: Gateway Orchestrator
Network hops per request	3 (W1-W2, W2-W3, W3-Backend)	1 (only W2 subprocess)
W3 requires HTTP server?	Yes (must accept C++ HTTP calls)	No (in-memory Java call)
Short-circuit on W1 block	No (request still hits W2)	Yes (skip W2 and W3 entirely)
Error handling location	Distributed across 3 components	Centralized in gateway
Testability of W2	Requires W3 mock running	Pure stdin/stdout subprocess
Coupling between worklets	W2 knows about W3 (callback)	Worklets know only the gateway contract
Logs location	Scattered across 3 processes	Single gateway log stream
Replace W2 with new impl	Update W1 and W3 callers	Update only gateway spawn line

The most consequential row in the table above is the second one. Under the chained pipeline, W3 - a Java class - must run its own HTTP server purely so that the C++ W2 can invoke it. This is absurd because W3 is a stateful workflow validator with no need for an external network interface. The orchestrator pattern eliminates this requirement entirely: W3 stays a plain Java class, called in-memory by the gateway.

3. Required Code Changes - File by File

The table below lists every file in the W2 codebase and the action required for the migration. Files marked KEEP are the core CYK engine and must remain unchanged. Files marked DELETE are architectural artifacts of the chained pipeline that have no role in the orchestrator pattern. Files marked MODIFY need their public API simplified. Files marked ADD are new entry points required for subprocess invocation.

File	Action	Reason
Token.hpp	KEEP	Defines lexical tokens produced by W1 - input contract
ValidationResult.hpp	MODIFY	Remove complex struct, return bool only
Grammar.hpp / .cpp	KEEP	CNF grammar engine - core CYK infrastructure
CnfConverter.hpp	KEEP	Grammar-to-CNF transformer - mathematical core
GrammarConfig.hpp / .cpp	KEEP	Loads grammar rules from cfg_rules.json
cfg_rules.json	KEEP	External grammar definition - no changes needed
Rulematch.hpp	KEEP	CYK back-tracing data - internal to CYK engine
Cell.hpp / .cpp	KEEP	CYK table cell - internal to CYK engine
CYKTable.hpp / .cpp	KEEP	O(n^3) dynamic programming parser - W2's heart
StructuralValidator.hpp	MODIFY	Remove action extraction step from pipeline
StructuralValidator.cpp	MODIFY	Stop calling ActionExtractor::extractPairs()
Worklet2Gateway.hpp	DELETE	Replaced by w2_main.cpp
Worklet2Gateway.cpp	DELETE	Contained W3 callback - no longer needed
test_pipeline.cpp	MODIFY	Update tests to new bool-only API
ActionExtractor.*	DELETE	Action extraction moves to Java gateway
TreeBuilder.*	KEEP	Used internally for depth checks (optional)
TreeChecker.*	KEEP	Tag matching and depth validation - security pipeline
ParseTreeNode.hpp	KEEP	Internal CYK data structure
w2_main.cpp	ADD	New standalone executable entry point

Summary: 11 files KEEP unchanged (CYK core), 4 files MODIFY (boundary simplification), 3 files DELETE (orchestration artifacts), 1 file ADD (subprocess entry point).

4. Change 1 - Remove W3 Callback from Worklet2Gateway

The most critical change is removing the Worklet3Delegate callback from Worklet2Gateway. In the current architecture, W2 holds a function pointer to W3 and invokes it directly after extracting the action. This couples W2 to W3's existence and forces W3 to expose an HTTP API if it is in a different process. Under the orchestrator pattern, W2 must not know that W3 exists - it simply validates structure and returns true or false.

BEFORE (Current)

```
// Worklet2Gateway.hpp - OLD
class Worklet2Gateway {
public:
    using Worklet3Delegate =
        std::function<bool(const std::string&,
                           const std::string&)>;

    void setWorklet3Callback(
            Worklet3Delegate cb) {
        worklet3Callback_ = std::move(cb);
    }

    ValidationResult validateAndProcess(
        const std::string& sessionId,
        const std::string& httpMethod,
        const std::vector<Token>& tokens) {
        // ... validate ...
        bool ok = worklet3Callback_(
            sessionId, action);
        if (!ok) result.reason =
            RejectReason::DOWNSTREAM_REJECTED;
        return result;
    }
private:
    Worklet3Delegate worklet3Callback_;
};
```

AFTER (Target)

```
// w2_main.cpp - NEW
// (replaces Worklet2Gateway entirely)
#include "StructuralValidator.hpp"

int main() {
    // Read token JSON from stdin
    std::string json;
    std::getline(std::cin, json);
    auto tokens = parseTokensJson(json);

    // Load grammar
    Grammar g;
    GrammarConfig::loadDefaultRules(g);

    // Validate structure ONLY
    StructuralValidator v(g, 3);
    bool ok = v.validate(tokens).success;

    // Output: just true or false
    std::cout << (ok ? "true" : "false")
              << std::endl;
    std::cout.flush();
    return 0;
}
```

Why this matters: After this change, W2 has zero knowledge of W3. The Java gateway calls W3 in-memory after receiving the boolean from W2's stdout. W3 never needs to expose an HTTP endpoint.

5. Change 2 - Remove Action Extraction & sessionId Handling

W2 currently extracts semantic key-value pairs from the JSON payload (e.g. `"action": "CART"`), prepends the HTTP method to produce `"POST_CART"`, and passes this action string along with the sessionId to W3. In the new architecture, W2 must NOT do this. The Java gateway already has the HTTP method and URL - it derives the action string itself via a Java helper class (`ActionExtractor`).

BEFORE (Current)

```
// StructuralValidator.cpp - OLD
ValidationResult
StructuralValidator::validate(
    const std::vector<Token>& tokens) const {
    // ... CYK validation ...
    // Extract action from parse tree:
    auto pairs = tree::ActionExtractor::
        extractPairs(treeRoot);
    result.extractedPairs = pairs;
    if (pairs.count("action")) {
        result.action = pairs["action"];
    }
    return result;
}

// Worklet2Gateway.cpp - OLD
std::string action = httpMethod + "_" +
                     result.action;
result.sessionId = sessionId;
```

AFTER (Target)

```
// StructuralValidator.cpp - NEW
ValidationResult
StructuralValidator::validate(
    const std::vector<Token>& tokens) const {
    // ... CYK validation ...
    // NO action extraction.
    // NO key-value pair extraction.
    // Just validate structure and depth.
    result.success = true;
    return result;
}

// w2_main.cpp - NEW
// No httpMethod parameter.
// No sessionId parameter.
// No action formatting.
// Just: validate(tokens) -> bool
```

Why this matters: W2 is now a pure structural validator. It receives tokens, verifies they form valid JSON, enforces depth and token-count limits, and returns true/false. No semantic extraction. The Java gateway derives the action from the HTTP method + URL (e.g. `POST + /api/login -> "POST_LOGIN"`) - this is faster (string comparison vs parse tree traversal) and avoids cross-language JSON conversion.

6. Change 3 - Simplify ValidationResult to Boolean Output

The current ValidationResult struct carries seven fields, most of which exist solely to communicate with W3 or carry forward data that the gateway already has. After the migration, W2 has no consumer for these fields - the gateway only needs to know whether the structure is valid. The struct should be reduced to a minimal shape, and the public API should expose a single boolean-returning function.

BEFORE (Current)

```
// ValidationResult.hpp - OLD
struct ValidationResult {
    bool success;
    RejectReason reason;
    std::string sessionId;
    std::string action;
    std::map<std::string,
             std::string> extractedPairs;
    std::shared_ptr<ParseTreeNode> tree;
    std::string rejectReason;
};

// 6 fields, 2 enums, complex to
// serialize across languages.
```

AFTER (Target)

```
// ValidationResult.hpp - NEW (minimal)
struct ValidationResult {
    bool success;
    std::string errorMessage;
    // For internal logging only.
    // Not exposed to gateway.
};

// Public API becomes:
bool validateTokens(
    const std::vector<Token>& tokens);

// Returns true/false.
// That is the entire contract.
```

Why this matters: A boolean is trivially serializable across any language boundary (stdin/stdout text "true" / "false"). The old struct required a JSON serializer in C++ and a JSON parser in Java - both sides had to agree on field names, types, and enum values. Eliminating the struct eliminates an entire class of integration bugs.

7. Change 4 - Create Standalone Executable (w2_main.cpp)

W2 must become a standalone executable that the Java gateway can spawn as a subprocess. The executable reads a JSON token stream from stdin, runs the CYK validator, and writes either "true" or "false" to stdout. The complete entry point is shown below - it is short because all the heavy lifting happens inside the existing StructuralValidator.

```
// w2_main.cpp - NEW FILE
// Entry point for subprocess invocation by Java gateway.
//
// Protocol:
//   stdin  : one line of JSON containing the token stream
//   stdout : "true\n" or "false\n"
//   exit 0 : normal completion (regardless of validation result)
//   exit 1 : fatal error (e.g. JSON parse failure)

#include "StructuralValidator.hpp"
#include "GrammarConfig.hpp"
#include "Grammar.hpp"
#include "Token.hpp"
#include <iostream>
#include <string>
#include <vector>

// Minimal JSON token parser (replace with your preferred lib).
// Reads tokens in the format produced by W1:
//   {"tokens":[{"type":"LBRACE","value":"{"}, ...]}
static std::vector<Token>
parseTokensJson(const std::string& json);

int main() {
    // 1. Read JSON tokens from stdin
    std::string json;
    std::getline(std::cin, json);
    if (json.empty()) {
        std::cout << "false" << std::endl;
        return 1;
    }

    // 2. Parse JSON -> vector<Token>
    std::vector<Token> tokens;
    try {
        tokens = parseTokensJson(json);
    } catch (...) {
        std::cout << "false" << std::endl;
        return 1;
    }

    // 3. Load CNF grammar (default rules from cfg_rules.json)
    Grammar grammar;
    GrammarConfig::loadDefaultRules(grammar);

    // 4. Run structural validation
    //    - Token limit (500) prevents JSON bomb DoS
    //    - CYK algorithm verifies grammatical validity
    //    - Depth check prevents recursive nesting attacks
    StructuralValidator validator(grammar, /*maxDepth=*/3);
    ValidationResult result = validator.validate(tokens);

    // 5. Output boolean - that's all the gateway needs
    std::cout << (result.success ? "true" : "false")
              << std::endl;
    std::cout.flush();   // CRITICAL: flush or Java blocks forever

    return 0;
}
```

Critical implementation notes:

- **Flush stdout.** Without `std::cout.flush()`, the Java ProcessBuilder will block indefinitely waiting for output that sits in the C++ buffer. This is the #1 bug in subprocess integration.

- **Use `std::getline`, not `std::cin >> json`.** The >> operator stops at whitespace; getline reads the entire line.

- **Exit 0 even on validation failure.** A non-zero exit code is interpreted by ProcessBuilder as a crash. Validation failure is a normal outcome, not an error.

- **Wrap parse errors in try/catch.** If the JSON from W1 is malformed, output "false" and exit 0 - do not let an exception terminate the process.

- **Single-line output only.** The Java caller uses `readLine()`, so any extra newlines will confuse the protocol.

8. Integration Contract - stdin / stdout Protocol

The integration contract between the Java gateway and the W2 C++ executable is a simple stdin/stdout protocol. There is no HTTP, no sockets, no shared memory. The gateway spawns the executable once per request, writes a single line of JSON to its stdin, and reads a single line ("true" or "false") from its stdout.

Input Format (Java -> W2 stdin)

A single line of JSON containing the token stream produced by W1. The format must match the JSON serialization of W1's `Worklet1Result.tokens()` list:

```
{
  "tokens": [
    {"type": "LBRACE",  "value": "{",   "position": 0},
    {"type": "KEY",     "value": "action", "position": 1},
    {"type": "COLON",   "value": ":",   "position": 7},
    {"type": "STRING",  "value": "CART", "position": 8},
    {"type": "RBRACE",  "value": "}",   "position": 14}
  ]
}
```

Output Format (W2 stdout -> Java)

A single line containing either the literal string "true" or "false", followed by a newline. No other output is permitted on stdout (debug logs must go to stderr).

Validation Result	stdout Output	Exit Code
Valid JSON structure	true	0
Malformed JSON (CYK rejects)	false	0
Token limit exceeded (>500)	false	0
Depth limit exceeded (>3)	false	0
Unbalanced braces/brackets	false	0
Empty input (no tokens)	false	0
JSON parse error in W2	false	0
Fatal W2 crash	(no output)	non-zero

Example Exchange

BEFORE (Current) **AFTER (Target)**

```
// Java gateway side
ProcessBuilder pb =
    new ProcessBuilder("./w2_parser");
Process p = pb.start();

// Write tokens JSON to stdin
p.getOutputStream().write(
    tokensJson.getBytes());
p.getOutputStream().close();

// Read boolean from stdout
String result = new BufferedReader(
    new InputStreamReader(
        p.getInputStream()))
    .readLine();

boolean isValid =
    "true".equals(result);
p.waitFor();
```

```
// C++ W2 side (w2_main.cpp)
int main() {
    std::string json;
    std::getline(std::cin, json);

    auto tokens =
        parseTokensJson(json);

    StructuralValidator v(g, 3);
    bool ok = v.validate(tokens)
                  .success;

    std::cout
        << (ok ? "true"
               : "false")
        << std::endl;
    std::cout.flush();
    return 0;
}
```

9. Java Gateway Caller - Reference Implementation

The W3 / gateway team will write the Java side of the subprocess call. The reference implementation is provided here so the W2 team knows exactly how their executable will be invoked. Note that this code lives in the Java gateway - W2 does not need to write any Java code.

```
package com.wafgateway.worklet3;

import java.io.*;
import java.util.concurrent.*;

/**
 * Calls the W2 C++ executable as a subprocess.
 * Writes token JSON to stdin, reads boolean from stdout.
 *
 * This is what the gateway calls between W1 and W3.
 */
public class W2SubprocessClient {

    private static final String W2_PATH = "./w2_parser";
    private static final int TIMEOUT_MS = 5000;

    /**
     * Validate a token stream via the W2 C++ executable.
     *
     * @param tokensJson  JSON from W1: {"tokens": [...]}
     * @return true if W2 says the structure is valid
     */
    public boolean validateTokens(String tokensJson)
            throws IOException, InterruptedException {

        ProcessBuilder pb = new ProcessBuilder(W2_PATH);
        pb.redirectErrorStream(false);  // stderr separate for debugging
        Process process = pb.start();

        // Write tokens to stdin
        try (OutputStream os = process.getOutputStream()) {
            os.write(tokensJson.getBytes("UTF-8"));
            os.flush();
        }
        // stdin MUST be closed or W2's getline blocks

        // Read result from stdout with timeout
        ByteArrayOutputStream baos = new ByteArrayOutputStream();
        Future<?> read = Executors.newSingleThreadExecutor().submit(() -> {
            try (InputStream is = process.getInputStream()) {
                byte[] buf = new byte[1024];
                int n;
                while ((n = is.read(buf)) > 0) baos.write(buf, 0, n);
            } catch (IOException ignored) {}
        });

        boolean finished = process.waitFor(TIMEOUT_MS, TimeUnit.MILLISECONDS);
        if (!finished) {
            process.destroyForcibly();
            return false;  // timeout = treat as invalid
        }
        read.get(500, TimeUnit.MILLISECONDS);

        String output = baos.toString("UTF-8").trim();
        return "true".equals(output);
    }
}
```

Key implementation details:

- **Close stdin after writing.** If stdin is not closed, the C++ `std::getline` call blocks waiting for more input.

- **Use a timeout.** A misbehaving W2 must not hang the entire gateway. 5 seconds is generous - normal validation takes less than 100ms.

- **Read stdout in a separate thread.** If the C++ process fills its stdout buffer while Java is still writing to stdin, both processes will deadlock.

- **Trim and exact-match the output.** Use `"true".equals(output)` not `output.contains("true")` - "false" contains "true" as a substring.

- **Redirect stderr separately.** W2 may write debug logs to stderr; these should be visible in the gateway log but must not pollute the boolean output channel.

10. Migration Steps and Timeline

The migration is broken into four phases, each with a clear deliverable and verification criterion. Phases must be executed in order - skipping ahead will leave the codebase in an inconsistent state. Total estimated effort: **3 working days**.

Phase	Tasks	Deliverable	Effort
1. Remove W3 Callback	Delete Worklet2Gateway.hpp/cpp. Remove Worklet3Delegate typedef. Remove setWorklet3Callback(). Update test_pipeline.cpp to drop callback setup.	W2 compiles and links with no W3 references. Existing CYK tests still pass.	1 day
2. Simplify Output	Reduce ValidationResult to {bool, string}. Remove sessionId, action, extractedPairs, tree fields. Remove ActionExtractor::extractPairs() call from validator. Update internal logging to use errorMessage field.	Validator returns minimal struct. No semantic data leaves W2.	0.5 day
3. Create w2_main.cpp	Write new main() function. Implement minimal JSON token parser (or use nlohmann/json). Wire stdin -> parse -> validate -> stdout. Add stdout.flush() call. Compile to ./w2_parser executable.	Executable runs standalone. echo '{"tokens":[]}' | ./w2_parser returns "false".	0.5 day
4. Subprocess Test	W3 team provides Java caller. Run integration tests: valid JSON, malformed, JSON bomb. Verify timeout handling. Verify stdin close behavior. Cross-check stderr logging.	Java can call ./w2_parser and receive correct boolean. All acceptance tests pass.	1 day

11. What to Keep and What to Delete

The single most important message of this guide: **the CYK engine is excellent and must remain unchanged.** W2's core mathematical infrastructure - the CNF grammar, the O(n^3) dynamic programming parser, the structural validator pipeline (token limits, depth checks) - is exactly what a production WAF needs. Only the orchestration boundary changes.

Component	Action	Rationale
CYKTable (O(n^3) DP parser)	KEEP	Mathematical core - the entire reason W2 exists
Grammar + CNF rules	KEEP	Chomsky Normal Form definitions are correct and tested
GrammarConfig (JSON loader)	KEEP	Loads cfg_rules.json - works perfectly
CnfConverter	KEEP	Transforms CFG to CNF - no changes needed
StructuralValidator::preCheck()	KEEP	500-token limit prevents JSON bomb DoS
StructuralValidator::validate() core	KEEP	CYK + depth + tag match pipeline
TreeChecker (depth, tag balance)	KEEP	Security pipeline - prevents structural attacks
TreeBuilder (parse tree)	KEEP	Required for depth calculation
Token.hpp	KEEP	Input contract with W1
Worklet2Gateway (callback dispatch)	DELETE	Chained-pipeline artifact - replaced by w2_main.cpp
Worklet3Delegate typedef	DELETE	W2 must not know W3 exists
ActionExtractor (semantic extraction)	DELETE	Action extraction moves to Java gateway
ValidationResult complex fields	DELETE	Replaced by bool + errorMessage
sessionId / httpMethod parameters	DELETE	Gateway has these in Java already
test_pipeline.cpp callback tests	MODIFY	Remove callback setup, update assertions
w2_main.cpp	ADD	New standalone subprocess entry point

Rule of thumb: If a file touches CYK, grammar, or structural security, keep it. If it touches W3, sessionId, action extraction, or callback dispatch, delete it. The CYK engine is W2's value - the orchestration scaffolding around it is what changes.

12. Testing and Acceptance Criteria

Before the W2 team declares the migration complete, the following acceptance tests must pass. Each test is invoked by piping a JSON token stream into the W2 executable and verifying the stdout output. The Java gateway team will run the same tests via the subprocess caller.

Test ID	Type	Test -> Expected Result
W2-1	Functional	Valid JSON object { "action": "CART" } -> stdout = "true"
W2-2	Functional	Valid nested JSON { "user": { "id": 101 } } -> stdout = "true"
W2-3	Security	Malformed JSON { "key": "value" (missing close) -> stdout = "false"
W2-4	Security	JSON bomb: 600 LBRACE tokens -> stdout = "false" (token limit)
W2-5	Security	Deeply nested 5-level JSON -> stdout = "false" (depth limit)
W2-6	Security	Unbalanced brackets [1, 2, 3 -> stdout = "false"
W2-7	Edge case	Empty input (no tokens) -> stdout = "false", exit 0
W2-8	Edge case	Unknown token type (e.g. FOO) -> stdout = "false"
W2-9	Robustness	Malformed JSON input (not even valid JSON) -> stdout = "false", exit 0
W2-10	Robustness	Process exits within 5 seconds for any input (no hangs)
W2-11	Integration	Java ProcessBuilder can spawn, write, read, get boolean
W2-12	Integration	stdin close behavior: Java closes stdin -> W2 produces output

Verification Procedure

The W2 team can verify each test manually before integration. For example, to run test W2-1 on Linux or macOS:

```
# Build the executable
g++ -std=c++17 -o w2_parser \
    w2_main.cpp StructuralValidator.cpp \
    CYKTable.cpp Cell.cpp Grammar.cpp \
    GrammarConfig.cpp

# Test W2-1: valid JSON
echo '{"tokens":[{"type":"LBRACE","value":"{"},\
{"type":"KEY","value":"action"},\
{"type":"COLON","value":":"},\
{"type":"STRING","value":"CART"},\
{"type":"RBRACE","value":"}"}]}' \
    | ./w2_parser
# Expected output: true

# Test W2-4: JSON bomb (600 braces)
python3 -c "print('{\"tokens\":[' + \
  ','.join(['{\"type\":\"LBRACE\",\"value\":\"{\"}']*600) + \
  ']}')" | ./w2_parser
# Expected output: false  (token limit exceeded)
```

Definition of Done

- All 12 acceptance tests pass with the expected output.
- No W3 references remain in the W2 codebase (grep for Worklet3, callback, W3 returns 0 hits).
- No action extraction code remains (grep for extractPairs, httpMethod returns 0 hits).
- The executable ./w2_parser runs standalone with no command-line arguments.
- Process exits within 5 seconds for any input, valid or invalid.
- Java gateway can call the executable and receive the correct boolean.

End of Guide. For questions about the integration contract or the Java caller implementation, contact the W3 / gateway team. For questions about the CYK engine or grammar, the existing W2 team owns those components and they remain unchanged.